TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

BASE DE DATOS II

UNIDAD 5 · CLASE 2 de 2

# Amenazas y protección de datos personales

*Administración y Seguridad aplicadas al proyecto integrador Food Store*

| | |
|---|---|
| **Asignatura** | Base de Datos II, Unidad 5: Administración y Seguridad |
| **Ubicación** | Semana 9 de la cursada, Clase 2 de 2 |
| **Requisito previo** | Clase 1: esquema de roles y permisos de Food Store (roles.sql) |
| **Motor** | PostgreSQL 16 o superior |
| **Meta de la clase** | Reconocer y mitigar las tres amenazas centrales sobre una base relacional, y aplicar los principios de protección de datos personales al proyecto |
| **Ciclo lectivo** | 2026 |

Apunte teórico de cátedra ·

La Clase 1 construyó un modelo de identidades y permisos: roles por función, mínimo privilegio, y una primera política de auditoría. Esta clase analiza qué ocurre cuando ese modelo falla o cuando es atacado deliberadamente. Se estudian las tres amenazas que el programa de la materia identifica como centrales (inyección de SQL, escalamiento de privilegios y fuga de datos), con un ejemplo trabajado de cada una sobre el esquema de Food Store.

La segunda mitad de la clase se ocupa de un problema distinto, aunque transversal a toda la materia: la protección de los datos personales que la base contiene. Se presentan los aspectos generales de la normativa vigente, los principios de diseño que se derivan de ella y las técnicas que permiten seguir trabajando, incluso con inteligencia artificial, sin exponer datos reales de personas. La clase cierra con la frontera concreta que esta cátedra traza para el uso de IA sobre datos de producción.

**Advertencia de alcance**
Esta clase enseña criterio técnico de administración de bases de datos, no derecho. Las referencias a la **Ley 25.326** y al Reglamento General de Protección de Datos europeo (RGPD) son nociones orientativas para el trabajo del DBA; ninguna decisión con implicancias legales reales debe tomarse sin la intervención de un profesional del derecho.

## 1. Inyección de SQL

### 1.1. Qué es y por qué sigue vigente

La inyección de SQL ocurre cuando una aplicación construye una sentencia SQL incorporando datos provistos por quien la usa (un formulario, un parámetro de URL) como si fueran parte confiable del texto del comando, en lugar de tratarlos como un valor. Quien controla ese dato de entrada puede entonces alterar el significado de la sentencia que finalmente se ejecuta. A pesar de ser una de las vulnerabilidades mejor documentadas de la industria del software, y de tener una solución simple y conocida desde hace décadas, sigue apareciendo entre las vulnerabilidades más frecuentes en aplicaciones reales, según las sucesivas ediciones del OWASP Top 10 (*MITRE Corporation, s.f.; OWASP Foundation, 2025*).

### 1.2. Anatomía de un ataque

Food Store necesita, en algún punto de su capa de acceso, verificar las credenciales de un usuario. Una implementación ingenua, que construye el SQL concatenando strings, se ve así:

*Versión vulnerable (no debe utilizarse)*

```
-- Dentro de una función que arma SQL dinámico, o en el código
-- de la aplicación que envía el texto ya armado al motor:
consulta := 'SELECT id, rol FROM usuario '
          || 'WHERE mail = ''' || p_mail || ''' '
          || 'AND contrasena = ''' || p_pass || ''';';
EXECUTE consulta INTO v_id, v_rol;
```

Si quien completa el formulario de login escribe, en el campo de correo, el valor ' OR '1'='1' -- y deja cualquier cosa en la contraseña, la variable consulta termina conteniendo:

```
SELECT id, rol FROM usuario WHERE mail = '' OR '1'='1' --' AND
contrasena = '...';
```

El -- convierte en comentario todo lo que sigue, incluida la verificación de la contraseña, y la condición OR '1'='1' es verdadera para cualquier fila. La sentencia deja de preguntar «¿existe un usuario con este mail y esta contraseña?» y pasa a devolver el primer usuario de toda la tabla, sin haber probado ninguna credencial válida. Si ese primer usuario resulta ser una cuenta con rol ADMIN, el atacante obtuvo acceso administrativo sin conocer una sola contraseña real.

### 1.3. Prevención: sentencias preparadas

La solución no consiste en filtrar comillas ni en escapar caracteres especiales a mano, ya que ambas son mitigaciones parciales que suelen omitirse en algún punto del código. La solución consiste en no

concatenar nunca el dato del usuario dentro del texto del comando: el valor viaja separado, como parámetro, y el motor lo trata siempre como un dato, jamás como código (*OWASP Foundation, s.f.-d*).

*Versión segura, con parámetro*

```
SELECT id, rol
INTO   v_id, v_rol
FROM   usuario
WHERE  mail = p_mail
AND    contrasena = crypt(p_pass, contrasena);
```

Al usarse directamente los parámetros p_mail y p_pass de la función, en lugar de concatenarlos como texto, PL/pgSQL los trata como valores tipados, nunca como fragmentos de SQL. Ni ' OR '1'='1' -- ni ninguna otra cadena puede alterar la estructura de la sentencia: en el peor de los casos, se busca sin éxito un usuario cuyo mail sea, literalmente, el texto completo ' OR '1'='1' --.

Cuando el SQL sí necesita construirse de forma dinámica, por ejemplo para armar un filtro opcional, la misma protección se logra con USING, que separa explícitamente el texto de la sentencia de sus parámetros (PostgreSQL Global Development Group, 2023a):

```
EXECUTE 'SELECT id, nombre FROM producto WHERE categoria_id = $1'
    INTO v_resultado
    USING p_categoria_id;
```

**Regla principal de la sección**
El valor que provee quien usa el sistema nunca se concatena al texto de una sentencia SQL. Siempre viaja separado, como parámetro. Esta única regla, aplicada sin excepciones, elimina la inyección de SQL como vector de ataque.

### 1.4. Defensa en profundidad: el mínimo privilegio como mitigación adicional

Ninguna defensa es absoluta, y por eso la seguridad se piensa en capas. Supóngase que, a pesar de todo, una inyección logra ejecutarse. Si la conexión atacada usa el rol app_web diseñado en la Clase 1 (sin DELETE amplio, sin TRUNCATE y sin privilegios de DDL), un intento de DROP TABLE usuario; inyectado en la misma cadena falla con el error «permission denied for table usuario». El mínimo privilegio no evita la inyección, pero reduce considerablemente lo que una inyección exitosa puede lograr. De este modo, lo trabajado en la Clase 1 y lo visto en esta clase constituyen dos capas de defensa independientes, cada una útil incluso si la otra falla.

### 1.5. Ejemplo trabajado: la función de login de Food Store

*Versión final, recomendada*

```
CREATE OR REPLACE FUNCTION fn_autenticar(p_mail TEXT, p_pass TEXT)
RETURNS TABLE(id BIGINT, rol rol) AS $$
BEGIN
    RETURN QUERY
    SELECT u.id, u.rol
    FROM   usuario u
    WHERE  u.mail = p_mail
    AND    u.contrasena = crypt(p_pass, u.contrasena)
    AND    u.eliminado = FALSE;
END;
$$ LANGUAGE plpgsql SECURITY INVOKER;
```

- **`p_mail y p_pass`** son parámetros tipados de la función: nunca se concatenan, viajan siempre como valores.
- **`crypt(p_pass, u.contrasena)`** recalcula el hash de la contraseña ingresada usando la sal ya almacenada en u.contrasena, y compara ambos hashes; la contraseña real nunca se compara ni se guarda en texto plano (requiere la extensión pgcrypto) (PostgreSQL Global Development Group, 2023c).
- **`AND u.eliminado = FALSE`** aplica el mismo patrón de borrado lógico del resto del proyecto: un usuario dado de baja no puede autenticarse.
- **`SECURITY INVOKER`** (el valor por omisión, explicitado aquí a propósito) hace que la función se ejecute con los privilegios de quien la llama, no de quien la creó; se contrasta con SECURITY DEFINER en la sección 2 (PostgreSQL Global Development Group, 2023b).

## 2. Escalamiento de privilegios

### 2.1. Qué es

El escalamiento de privilegios ocurre cuando una identidad obtiene, por una vía no prevista, más capacidad de la que se le otorgó. Se distinguen dos direcciones: el escalamiento vertical, cuando un usuario de bajo privilegio obtiene privilegios de administrador; y el escalamiento horizontal, cuando un usuario accede a datos o funciones de otro usuario de su mismo nivel, por ejemplo, un cliente de Food Store que logra ver los pedidos de otro cliente.

### 2.2. Vectores típicos en PostgreSQL

- **Funciones SECURITY DEFINER mal configuradas.** Una función SECURITY DEFINER se ejecuta con los privilegios de quien la creó, no de quien la invoca. Si cualquier rol tiene EXECUTE sobre una

función así, y esa función hace algo sensible, ese rol «hereda» temporalmente el privilegio de su dueño mientras la función corre.

- **search_path sin fijar dentro de una función SECURITY DEFINER.** Un atacante con permiso de crear objetos en algún esquema puede definir una tabla o función con el mismo nombre que una que la función privilegiada usa internamente, en un esquema que el search_path resuelve antes que el correcto. La función SECURITY DEFINER termina ejecutando el objeto falso, con privilegios elevados. Este vector está documentado como la vulnerabilidad CVE-2018-1058 (PostgreSQL Global Development Group, 2018, 2023b).
- **Permisos por defecto demasiado abiertos.** Antes de PostgreSQL 15, todo rol nuevo recibía automáticamente CREATE sobre el esquema public, lo que habilita justamente el vector anterior si nadie lo revoca explícitamente (PostgreSQL Global Development Group, 2022, 2023d).
- **Membresía de roles otorgada de más.** Hacer a un rol miembro de otro «por las dudas», sin una necesidad concreta, es una forma poco visible de violar el mínimo privilegio de la Clase 1.

### 2.3. El caso de Food Store

El panel de soporte necesita poder resetear la contraseña de un usuario sin que rol_soporte tenga permiso de UPDATE directo sobre la tabla usuario, ya que eso implicaría otorgarle más privilegios de los necesarios. La solución habitual es una función SECURITY DEFINER, acotada, que hace exactamente esa tarea y nada más:

```
CREATE OR REPLACE FUNCTION fn_resetear_password(
    p_usuario_id BIGINT, p_nueva TEXT
) RETURNS VOID
SECURITY DEFINER
SET search_path = public, pg_temp
AS $$
BEGIN
    UPDATE usuario
    SET    contrasena = crypt(p_nueva, gen_salt('bf'))
    WHERE  id = p_usuario_id AND eliminado = FALSE;
END;
$$ LANGUAGE plpgsql;

REVOKE ALL ON FUNCTION fn_resetear_password FROM PUBLIC;
GRANT  EXECUTE ON FUNCTION fn_resetear_password TO rol_soporte;
```

- **`SECURITY DEFINER`** hace que la función corra con los privilegios de quien la creó (*el dueño del esquema, que sí puede modificar usuario*), y no con los de rol_soporte, que no podría.

- **`SET search_path = public, pg_temp`** es la mitigación específica del ataque de la sección 2.2: fija de forma explícita en qué esquemas busca sus objetos la función, sin depender del search_path de quien la invoca, que podría estar manipulado (PostgreSQL Global Development Group, 2023b).
- **`REVOKE ALL ... FROM PUBLIC`** retira el permiso de ejecución que el motor otorga por omisión a cualquier rol, y el GRANT posterior lo devuelve únicamente a rol_soporte: exactamente el mínimo privilegio de la Clase 1, aplicado a una función.
- **La función solo actualiza contrasena: no expone** ni modifica ninguna otra columna. Un poder amplio ("acceso de soporte a usuarios") queda reducido a la única operación concreta que ese rol necesita.

## 3. Fuga de datos

### 3.1. Vectores habituales

- **Backups sin cifrar y sin control de acceso.** Un respaldo completo de la base es, en muchos sentidos, un activo más sensible que la base en producción: contiene todo, y suele copiarse a ubicaciones con menos controles (un disco externo, un bucket de almacenamiento) que el servidor original. Se retoma en la Unidad 6.
- **Logs con datos sensibles.** Ya señalado en la Clase 1: un log que registra sentencias completas puede terminar conteniendo contraseñas o datos personales si esas sentencias los incluyen como literales (OWASP Foundation, s.f.-c).
- **Vistas que exponen más columnas de las necesarias.** Escribir SELECT * dentro de la definición de una vista, en lugar de listar explícitamente las columnas necesarias, es la forma más común de filtrar por descuido una columna sensible el día que alguien agrega una a la tabla base.
- **Mensajes de error demasiado detallados.** Devolver hacia un cliente no autenticado el texto completo de un error del motor puede revelar nombres de tablas, columnas o incluso fragmentos de la consulta original, información que facilita un ataque posterior (OWASP Foundation, s.f.-b).
- **Canales laterales.** Un formulario de login que responde más rápido cuando el mail no existe que cuando existe pero la contraseña es incorrecta permite, sin leer ningún dato directamente, enumerar qué correos están registrados (OWASP Foundation, s.f.-a).

### 3.2. Prevención

- Cifrado en tránsito: la conexión entre la aplicación y el motor viaja por TLS/SSL, no en texto plano por la red (PostgreSQL Global Development Group, 2023e).

- Cifrado en reposo: discos y, en particular, backups cifrados. Estas nociones se retoman en la Unidad 6.
- Necesidad de saber (need to know): cada vista y cada rol expone exactamente las columnas que su función requiere, ni una más. Es el mismo criterio de v_usuarios_publico (Unidad 3) aplicado de forma sistemática.
- Revisión periódica de vistas: cada vez que se agrega una columna a una tabla base, revisar si alguna vista con SELECT * (o equivalente) la está exponiendo sin que corresponda.
- Mensajes de error genéricos hacia el cliente («no se pudo procesar la solicitud»), con el detalle técnico completo solo en el log interno protegido.

## 4. Protección de datos personales

### 4.1. Normativa vigente (nociones)

En Argentina, la referencia central es la Ley 25.326 de Protección de los Datos Personales y su normativa reglamentaria (Ley 25.326, 2000). A nivel internacional, el Reglamento General de Protección de Datos de la Unión Europea (RGPD/GDPR; Reglamento [UE] 2016/679, 2016) es una de las referencias más citadas a nivel global, y buena parte de las prácticas de la industria, incluidas las de empresas que no operan en Europa, lo toman como estándar de facto. Ambos marcos comparten una lógica común: la persona a la que se refieren los datos conserva derechos sobre ellos (acceso, rectificación y supresión), y quien los administra tiene obligaciones específicas de cuidado, no solo la potestad técnica de almacenarlos.

### 4.2. Categorías de datos en Food Store

| Columna | Categoría | Por qué |
|---|---|---|
| usuario.mail | Dato personal | Identifica a la persona y permite contactarla. |
| usuario.celular | Dato personal | Identifica y permite un contacto directo. |
| usuario.nombre / apellido | Dato personal | Identificación directa de la persona. |
| usuario.contrasena | Credencial sensible | Debe almacenarse siempre con hash (sección 1.5), nunca en texto plano. |
| Datos de pago (número de tarjeta) | Dato sensible financiero | Food Store no los almacena; es una decisión de diseño correcta, no una omisión. |

### 4.3. Principios de diseño

- **Minimización.** Solo se recolecta el dato que efectivamente se va a usar. Food Store no pide documento de identidad ni dirección física porque ninguna funcionalidad del sistema los necesita. Un dato que nunca se almacenó no puede filtrarse (Ley 25.326, 2000; Reglamento [UE] 2016/679, 2016).
- **Finalidad.** Un dato se usa para lo que la persona aceptó al darlo, no para cualquier otro propósito que resulte conveniente después.
- **Conservación limitada.** Debe definirse por cuánto tiempo se conserva el mail de un usuario dado de baja. El borrado lógico del proyecto (eliminado = TRUE) preserva el historial de negocio, pero eso no implica que los datos de contacto deban conservarse para siempre sin un criterio explícito.

## 5. Anonimización y enmascaramiento (nociones)

### 5.1. Una distinción central

Anonimizar un dato es destruir de forma irreversible su vínculo con la persona a la que describía: una vez anonimizado, ya no es, técnicamente, un dato personal. Seudonimizar o enmascarar es reversible mediante una clave que se conserva por separado: el dato sigue siendo, a efectos de la protección de datos, un dato personal, pero el riesgo operativo de manejarlo baja considerablemente, porque quien accede al valor enmascarado no puede reconstruir el original sin esa clave (*Grupo de Trabajo del Artículo 29, 2014; Reglamento [UE] 2016/679, 2016*).

### 5.2. Técnicas

- **Hashing con sal.** Ya usado para contrasena (secciones 1.5 y 2.3): irreversible por diseño para ese caso puntual: ni siquiera el propio sistema puede recuperar la contraseña original, solo verificar una nueva contra el hash guardado.
- **Tokenización.** Reemplazar el valor real por un token sin relación matemática con el original, guardando la correspondencia en una tabla de mapeo protegida aparte, accesible solo a quien de verdad necesite revertirla.
- **Generalización.** En vez de una fecha de nacimiento exacta, guardar un rango etario; en vez de una dirección puntual, un barrio o código postal. Esta técnica reduce la precisión de un dato sin eliminarlo del todo, cuando el análisis que lo necesita no requiere esa precisión.
- **Data masking estático vs. dinámico.** El enmascaramiento estático genera una copia enmascarada de la base, pensada para ambientes de prueba (*sección 5.3*); el dinámico enmascara el dato real al vuelo, según quién consulta, sin duplicar la base, de forma similar a lo

que ya se vio con las vistas de seguridad y con RLS en la Clase 1 (*Grupo de Trabajo del Artículo 29, 2014; McCallister et al., 2010).*

### 5.3. El caso de Food Store: un dataset de prueba anonimizado

```
CREATE TABLE usuario_anon AS
SELECT id,
       'Usuario ' || id                      AS nombre,
       'Apellido ' || id                      AS apellido,
       'usuario' || id || '@ejemplo.test'     AS mail,
       NULL                                   AS celular,
       'hash_no_valido'                       AS contrasena,
       rol, eliminado, created_at
FROM   usuario;
```

- **`'Usuario ' || id`** / **`'Apellido ' || id`** sustituyen el nombre real por un valor sintético pero determinista: cada usuario sigue siendo distinguible de los demás, y las estadísticas de cardinalidad se conservan, algo que un valor fijo como 'Anónimo' para todas las filas rompería.
- **`'usuario' || id || '@ejemplo.test'`** conserva la unicidad del mail (necesaria si alguna consulta o índice depende de ella) sin que el valor corresponda a una persona real.
- **`NULL AS celular`** elimina directamente un dato que ninguna consulta de prueba necesita conservar.
- **`'hash_no_valido' AS contrasena`** reemplaza cualquier hash real por un valor que no corresponde a ninguna contraseña, lo que impide utilizarlo, incluso por error.

Sobre esta tabla, y sobre equivalentes para producto, pedido y detalle_pedido si fuera necesario, se puede seguir trabajando exactamente igual que en las unidades anteriores: medir índices, probar vistas, probar reportes. Ninguna fila contiene ya un dato real de una persona.

## 6. La frontera de la privacidad con la inteligencia artificial

### 6.1. Qué información jamás se envía a un asistente de IA

- Filas reales de la tabla usuario: mail, celular, nombre y apellido reales, y por supuesto la contraseña, aunque esté hasheada.
- Cualquier dato de pago, si el sistema llegara a incorporarlo en el futuro.
- Logs de auditoría sin anonimizar (sección 5.4 de la Clase 1).
- Backups completos de la base de producción.

### 6.2. Cómo anonimizar para poder consultar a la IA

La restricción de 6.1 no significa dejar de usar IA sobre estos temas: significa usarla sobre la base anonimizada de la sección 5.3, o sobre extractos de log donde el mail y el nombre fueron reemplazados por un identificador numérico. Las tareas realizadas en las unidades anteriores (solicitar a la IA que proponga índices, que explique un plan de EXPLAIN o que revise una vista) se pueden seguir realizando; cambia únicamente la fuente de datos sobre la que se trabaja.

### 6.3. Simulacro de incidente con IA

Se le entrega a la IA un extracto de log de auditoría, ya con los mails reemplazados por identificadores, y se le pide reconstruir qué ocurrió:

*Extracto de log entregado a la IA (anonimizado)*

```
2026-08-14 03:12:01 login fallido  usuario_id=None
origen=203.0.113.55
2026-08-14 03:12:04 login fallido  usuario_id=None
origen=203.0.113.55
2026-08-14 03:12:07 login fallido  usuario_id=None
origen=203.0.113.55
...  (340 intentos más, mismo origen, 03:12 a 03:19)
2026-08-14 03:19:52 login exitoso  usuario_id=00187
origen=203.0.113.55
2026-08-14 03:20:03  SELECT * FROM usuario  (00187 filas
devueltas)
2026-08-14 03:20:11  rol de usuario_id=04412 cambiado: USUARIO ->
ADMIN
```

*Reconstrucción esperada de la IA, y su validación humana*

Una respuesta razonable de la IA identifica un patrón de fuerza bruta (cientos de intentos fallidos desde el mismo origen en pocos minutos), seguido de un login exitoso (la contraseña se adivinó o se obtuvo de otra fuente), una lectura masiva de la tabla completa de usuarios y de un cambio de rol que otorga privilegios de administrador a otra cuenta: una fuga de datos seguida de un escalamiento de privilegios vertical, ya que una cuenta común termina con privilegios de administrador, mediante los mecanismos estudiados en las secciones 2 y 3.

**Validación humana obligatoria**
La reconstrucción de la IA es plausible, pero no se actúa sobre ella sin verificarla: se contrasta con los registros del servidor, con pg_stat_activity si la sesión sigue activa y con los backups de la fecha para confirmar el alcance real, se identifica si 203.0.113.55 corresponde a un origen conocido o no, y solo entonces se decide la contención (bloquear el origen, revertir el cambio de rol del usuario 04412, forzar el cambio de contraseña de la cuenta comprometida, y revisar si ese SELECT devolvió datos que efectivamente salieron del sistema). La IA agiliza la formulación de la hipótesis; la decisión y la responsabilidad corresponden al administrador.

## 7. Síntesis y cierre de la unidad

Las dos clases de esta unidad recorrieron el ciclo completo de la seguridad de una base de datos: primero, construir un modelo de identidades y permisos con mínimo privilegio y auditoría (Clase 1); después, reconocer cómo ese modelo puede fallar o ser atacado (inyección de SQL, escalamiento de privilegios y fuga de datos) y cómo se protege, de forma transversal, el dato personal que la base contiene, incluyendo la frontera concreta que esta cátedra traza para el trabajo con inteligencia artificial.

Los elementos construidos en ambas clases (roles.sql de la Clase 1, la función de autenticación segura, la función SECURITY DEFINER acotada y el dataset anonimizado de esta clase) son la base directa del laboratorio integrador de la unidad, que se distribuye a continuación como trabajo práctico: el esquema de roles y permisos probado con un simulacro de acceso indebido real sobre la base del proyecto. La Unidad 6 retoma varios de estos temas, como los backups cifrados y el monitoreo, desde la óptica del mantenimiento.

## Glosario de términos

**Inyección de SQL.** Vulnerabilidad que permite alterar el significado de una sentencia SQL insertando datos de entrada no tratados como parámetros.

**Sentencia preparada / parámetro.** Mecanismo por el cual un valor de entrada se envía al motor separado del texto del comando, de modo que nunca se interpreta como código.

**SECURITY DEFINER.** Atributo de una función que hace que se ejecute con los privilegios de quien la creó, no de quien la invoca.

**search_path.** Lista de esquemas en la que el motor busca un objeto nombrado sin calificar; fijarlo explícitamente en funciones SECURITY DEFINER previene un vector de escalamiento conocido.

**Escalamiento de privilegios.** Obtención de más capacidad de la otorgada; vertical (hacia un rol de más privilegio) u horizontal (hacia los datos de otra identidad del mismo nivel).

**Fuga de datos (data leakage).** Exposición de datos a quien no debería poder acceder a ellos, por un canal no previsto.

**Cifrado en tránsito / en reposo.** Protección de los datos mientras viajan por la red, o mientras están almacenados en disco, respectivamente.

**Ley 25.326.** Ley argentina de Protección de los Datos Personales.

**RGPD / GDPR.** Reglamento General de Protección de Datos de la Unión Europea; referencia internacional de facto.

**Minimización de datos.** Principio de recolectar únicamente el dato personal estrictamente necesario para la finalidad declarada.

**Anonimización.** Transformación irreversible que elimina el vínculo entre un dato y la persona a la que describía.

**Seudonimización / enmascaramiento.** Transformación reversible mediante una clave separada; el dato sigue siendo personal a efectos legales.

**Tokenización.** Sustitución de un valor real por un token sin relación matemática con el original.

**Data masking (estático / dinámico).** Enmascaramiento aplicado a una copia de la base (estático) o al vuelo según quién consulta (dinámico).

**Need to know.** Principio de exponer, a cada rol o vista, únicamente los datos que su función requiere.

## Referencias


Grupo de Trabajo del Artículo 29. (2014). *Opinion 05/2014 on anonymisation techniques* (WP216). Comisión Europea. https://ec.europa.eu/justice/article-29/documentation/opinion-recommendation/files/2014/wp216_en.pdf

Ley 25.326 de Protección de los Datos Personales. (2000). *Boletín Oficial de la República Argentina*. https://www.argentina.gob.ar/normativa/nacional/64790/texto

McCallister, E., Grance, T. y Scarfone, K. (2010). *Guide to protecting the confidentiality of personally identifiable information (PII)* (NIST Special Publication 800-122). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-122

MITRE Corporation. (s.f.). *CWE-89: Improper neutralization of special elements used in an SQL command (SQL injection)*. Common Weakness Enumeration. https://cwe.mitre.org/data/definitions/89.html

OWASP Foundation. (s.f.-a). *Authentication cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP Foundation. (s.f.-b). *Error handling cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

OWASP Foundation. (s.f.-c). *Logging cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

OWASP Foundation. (s.f.-d). *SQL injection prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation. (2025). *A05:2025 Injection*. OWASP Top 10:2025. https://owasp.org/Top10/2025/A05_2025-Injection/

PostgreSQL Global Development Group. (2018). *A guide to CVE-2018-1058: Protect your search path*. PostgreSQL Wiki. https://wiki.postgresql.org/wiki/A_Guide_to_CVE-2018-1058:_Protect_Your_Search_Path

PostgreSQL Global Development Group. (2022). *Release 15*. PostgreSQL release notes. https://www.postgresql.org/docs/release/15.0/

PostgreSQL Global Development Group. (2023a). *Basic statements* [PL/pgSQL]. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/plpgsql-statements.html

PostgreSQL Global Development Group. (2023b). *CREATE FUNCTION*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-createfunction.html

PostgreSQL Global Development Group. (2023c). *pgcrypto*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/pgcrypto.html

PostgreSQL Global Development Group. (2023d). *Schemas*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ddl-schemas.html

PostgreSQL Global Development Group. (2023e). *Secure TCP/IP connections with SSL*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ssl-tcp.html

Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (Reglamento General de Protección de Datos). (2016). *Diario Oficial de la Unión Europea, L 119*, 1-88. https://eur-lex.europa.eu/eli/reg/2016/679/oj